

Alumno: Diego Mora Márquez
Profesor: Luis de Jesús López Hernandez

Semestre 4

Mérida Yucatán **2026**

Rasterización vs. Ray Tracing

Investigación técnica y comparativa · Gráficas por Computadora · Proyecto Cenote Vivo

Introducción

Toda imagen generada por computadora, desde la interfaz de un teléfono hasta un fotograma de una película animada, es el resultado de resolver la misma pregunta: qué color debe tener cada píxel de la pantalla para que el conjunto se parezca a una escena tridimensional vista desde un punto concreto. Existen dos grandes familias de algoritmos para responderla. La **rasterización** proyecta la geometría sobre la pantalla y decide qué píxeles cubre cada triángulo; el **trazado de rayos** (*ray tracing*) recorre el camino inverso y lanza rayos desde la cámara hacia la escena para averiguar qué superficie ve cada píxel y cómo llega la luz hasta ella.

Este documento explica ambos paradigmas desde sus fundamentos, compara su funcionamiento, su costo y sus aplicaciones, y aborda dos temas que atraviesan a cualquier motor gráfico: el *aliasing* con sus técnicas de corrección y la diferencia entre renderizar en tiempo real o fuera de línea. A lo largo del texto se relaciona cada concepto con las decisiones tomadas en el prototipo móvil *Cenote Vivo*, desarrollado en Unity con Universal Render Pipeline (URP) para Android.

1. ¿Qué es el renderizado?

El renderizado es el proceso de generar una imagen bidimensional a partir de una descripción de una escena. Esa descripción incluye la **geometría** (normalmente mallas de triángulos), los **materiales** que indican cómo responde cada superficie a la luz, las **fuentes de luz** y una **cámara** con posición, orientación y campo de visión. El resultado es una matriz de píxeles —el *framebuffer*— que se envía al monitor o se guarda como archivo.

Desde el punto de vista físico, renderizar consiste en aproximar el transporte de la luz. Kajiya (1986) formalizó este problema en la **ecuación de renderizado**, que expresa la radiancia que sale de un punto en una dirección como la suma de la luz que ese punto emite más la integral de toda la luz que recibe desde el hemisferio que lo rodea, ponderada por las propiedades del material y por el ángulo de incidencia:

$$L_o(x, \omega) = L_e(x, \omega) + \int f_r(x, \omega_i, \omega) \cdot L_i(x, \omega_i) \cdot (\omega_i \cdot n)\, d\omega_i$$

Ningún algoritmo práctico resuelve esta integral de forma exacta. Cada técnica de renderizado es, en el fondo, una estrategia distinta para aproximarla: algunas sacrifican precisión física a cambio de velocidad y otras persiguen la fidelidad aunque cueste horas por imagen.

2. ¿Qué es la rasterización?

La rasterización es la técnica que convierte primitivas geométricas descritas con vectores —puntos, líneas y, sobre todo, triángulos— en un conjunto discreto de fragmentos alineados con la cuadrícula de píxeles de la pantalla. Su lógica está **orientada al objeto**: el algoritmo toma cada triángulo, lo proyecta sobre el plano de la imagen y determina qué píxeles quedan dentro de él. La visibilidad entre objetos se resuelve después mediante un búfer de profundidad (*Z-buffer*), que conserva para cada píxel solo el fragmento más cercano a la cámara.

Es el método que utilizan prácticamente todos los videojuegos, aplicaciones interactivas y sistemas de interfaz desde la década de 1990, porque se adapta de forma natural al hardware paralelo de las GPU: miles de triángulos y millones de fragmentos se procesan de manera independiente y simultánea.

3. Funcionamiento conceptual de la rasterización

El proceso se organiza como una cadena de etapas llamada **pipeline gráfico**. Cada etapa recibe el resultado de la anterior, lo transforma y lo entrega a la siguiente, de modo que mientras un triángulo se está sombreando, otros ya se están transformando.



Figura 1. Etapas principales del pipeline de rasterización. Las etapas resaltadas son programables mediante shaders.

3.1 Aplicación y envío de comandos

En la CPU, el motor decide qué objetos dibujar, prepara sus búferes de vértices e índices, fija el estado de la GPU (shader, texturas, modo de mezcla) y emite una **llamada de dibujo** (*draw call*)

por cada lote de geometría. Cada cambio de estado tiene un costo de CPU y de controlador; por eso los motores agrupan objetos que comparten material, como hace el *SRP Batcher* de Unity.

3.2 Transformación geométrica (Vertex Shader)

Cada vértice se multiplica por una serie de matrices de 4×4 en coordenadas homogéneas. La **matriz de modelo** lo lleva del espacio local del objeto al espacio del mundo; la **matriz de vista** lo expresa respecto a la cámara; y la **matriz de proyección** lo transforma al espacio de recorte (*clip space*), donde la perspectiva queda codificada en la componente *w*. El vertex shader también puede calcular normales, coordenadas de textura o animar la malla.

3.3 Ensamblado de primitivas y clipping

Los vértices transformados se agrupan en triángulos. Los que quedan totalmente fuera del volumen de visión (*frustum*) se descartan; los que lo cruzan se **recortan** contra sus planos, generando nuevos vértices en las intersecciones. Aquí también se aplica el *back-face culling*, que elimina los triángulos que dan la espalda a la cámara y suele ahorrar cerca de la mitad del trabajo en mallas cerradas.

3.4 División de perspectiva y transformación al viewport

Las coordenadas se dividen entre *w*, lo que produce las coordenadas normalizadas de dispositivo (NDC) en el rango [−1, 1]. Esta división es la que hace que los objetos lejanos se vean más pequeños. Después, la transformación de *viewport* escala esos valores a coordenadas de píxel y conserva la profundidad para la prueba de visibilidad.

3.5 Rasterización de primitivas

Con el triángulo ya en coordenadas de pantalla, el hardware determina qué muestras quedan cubiertas. El método habitual evalúa tres **funciones de arista** (*edge functions*): una muestra está dentro del triángulo si queda del mismo lado de sus tres aristas. Por cada muestra cubierta se genera un **fragmento** y sus atributos (color, UV, normal, profundidad) se interpolan con coordenadas baricéntricas corregidas por perspectiva.

3.6 Sombreado de fragmentos (Fragment Shader)

Cada fragmento ejecuta el fragment shader, que calcula su color final: muestrea texturas, evalúa el modelo de iluminación y aplica efectos como niebla o transparencia. Es la etapa más costosa en dispositivos móviles, porque su costo crece con la resolución de la pantalla y con el **overdraw** (píxeles dibujados varias veces).

3.7 Pruebas por fragmento y mezcla

Finalmente se aplican las pruebas de profundidad y de *stencil*. Si el fragmento sobrevive, su color se escribe en el framebuffer o se mezcla con el existente cuando el material es transparente. En las GPU móviles basadas en teselas (*tile-based*), estas operaciones ocurren en una memoria pequeña dentro del chip, lo que reduce el tráfico hacia la memoria principal.

4. ¿Qué es Ray Tracing?

El trazado de rayos es una técnica de renderizado **orientada a la imagen**: en lugar de llevar cada objeto hacia la pantalla, parte de cada píxel y pregunta qué objeto de la escena es visible a través de él. Para ello lanza un rayo desde la cámara, calcula su intersección con la geometría y, a partir del punto de impacto, lanza nuevos rayos hacia las luces o en la dirección de reflexión y refracción. Turner Whitted (1980) describió la versión recursiva clásica de este algoritmo, que ya producía de forma natural sombras nítidas, reflejos y refracciones.

Las variantes modernas, como el *path tracing*, lanzan muchos rayos aleatorios por píxel para estimar la ecuación de renderizado mediante integración de Monte Carlo. Así obtienen iluminación global, sombras suaves y cáusticas con un comportamiento físicamente plausible.

5. Funcionamiento conceptual del Ray Tracing

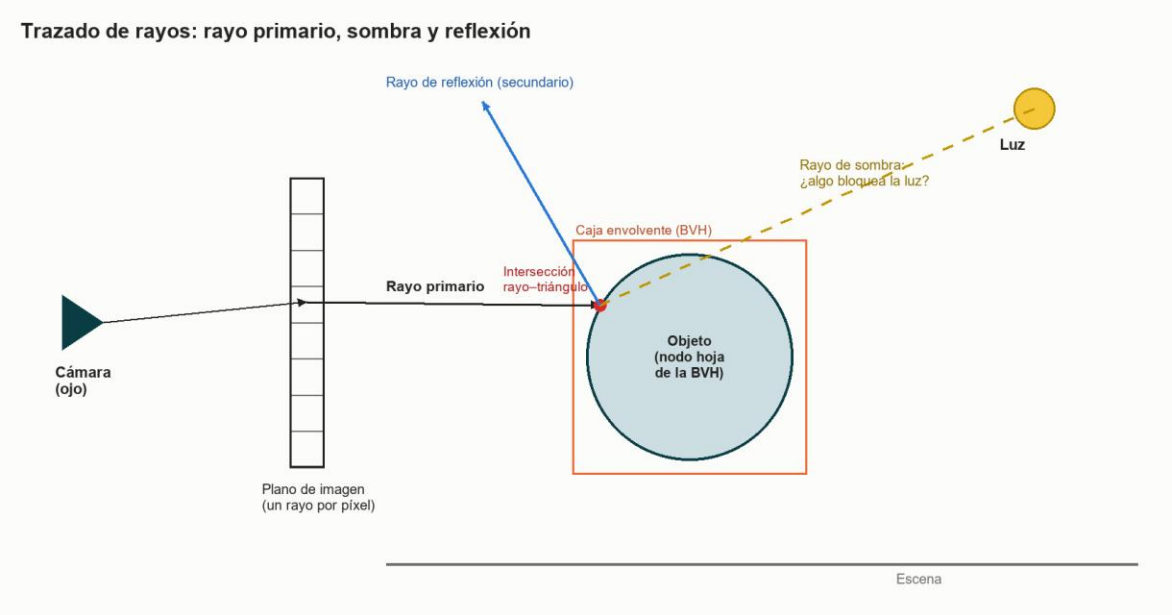


Figura 2. Esquema de un trazado de rayos tipo Whitted: rayo primario, prueba de sombra y rayo de reflexión.

5.1 Lanzamiento de rayos primarios

Por cada píxel se construye un rayo con origen en la cámara y dirección hacia el centro del píxel (o hacia puntos aleatorios dentro de él cuando se quiere suavizar los bordes). Matemáticamente un rayo es $P(t) = O + t \cdot D$, donde O es el origen, D la dirección normalizada y $t \geq 0$ la distancia recorrida.

5.2 Estructuras de aceleración: BVH

Probar cada rayo contra todos los triángulos sería prohibitivo: una escena de un millón de triángulos a 1080p exigiría billones de pruebas por imagen. Por eso la geometría se organiza en una **jerarquía de volúmenes envolventes** (*Bounding Volume Hierarchy*, BVH): un árbol cuyos nodos son cajas alineadas a los ejes que contienen a sus hijos. El rayo se compara primero contra la caja raíz y solo desciende por las ramas cuyas cajas atraviesa, lo que reduce el costo de la búsqueda de lineal a aproximadamente logarítmico respecto al número de primitivas. Construir y actualizar la BVH cuando los objetos se mueven es en sí mismo un costo importante.

5.3 Intersección rayo–malla

Al llegar a una hoja de la BVH se prueban los triángulos que contiene. El algoritmo de Möller–Trumbore resuelve la intersección con un sistema lineal pequeño que devuelve la distancia t y las coordenadas baricéntricas (u, v) del impacto; con ellas se interpolan la normal y las coordenadas de textura igual que en la rasterización. De todas las intersecciones válidas se conserva la más cercana.

5.4 Rayos secundarios: sombra, reflexión y refracción

Desde el punto de impacto se lanza un **rayo de sombra** hacia cada fuente de luz: si encuentra un obstáculo antes de llegar, el punto está en sombra para esa luz. Si el material es reflectante se lanza un **rayo de reflexión** con la dirección especular, y si es transparente un **rayo de refracción** desviado según la ley de Snell. Cada uno de estos rayos repite el proceso de forma recursiva hasta una profundidad máxima o hasta que su contribución es despreciable. En *path tracing* los rebotes se eligen al azar según la función de reflectancia del material, y el ruido resultante se reduce con más muestras o con filtros de eliminación de ruido (*denoisers*).

6. Diferencias fundamentales

Tabla 1. Comparación conceptual entre los dos paradigmas.

Aspecto	Rasterización	Ray Tracing
Orientación del algoritmo	Objeto → pantalla: cada triángulo se proyecta y se buscan los píxeles que cubre.	Pantalla → escena: cada píxel lanza rayos y busca qué superficie ve.
Visibilidad	Z-buffer por píxel.	Intersección más cercana a lo largo del rayo.
Información de la escena	Cada triángulo se procesa de forma aislada; no «ve» al resto de la escena.	Cada rayo puede consultar toda la escena (sombras, reflejos).
Efectos globales	Requieren técnicas aparte: shadow maps, reflection probes, SSR, lightmaps.	Surgen de forma natural con rayos secundarios.
Costo principal	Proporcional al número de triángulos y de fragmentos.	Proporcional a píxeles × rayos por píxel × costo del recorrido de la BVH.
Hardware	Unidades fijas de rasterización en toda GPU, incluidas las móviles.	Núcleos dedicados (RT Cores, Ray Accelerators) en GPU recientes; muy limitado en móviles.
Escenas dinámicas	Cambiar la geometría casi no tiene costo adicional.	Hay que reconstruir o reajustar la BVH.

La diferencia esencial es de *información*. La rasterización es extremadamente eficiente porque cada triángulo se dibuja sin consultar a los demás, pero precisamente por eso no puede saber, por sí sola, si un punto está a la sombra de otro objeto o qué se refleja en él. Cada uno de esos efectos se resuelve con una aproximación adicional. El trazado de rayos, al poder interrogar toda la escena, obtiene esos efectos de manera directa, a cambio de un patrón de acceso a memoria mucho menos predecible.

7. Ventajas y limitaciones

7.1 Rasterización

- **Ventajas:** velocidad muy alta y estable; soporte universal en hardware, desde relojes inteligentes hasta consolas; ecosistema maduro de técnicas (LOD, *culling*, *batching*) y herramientas; consumo energético contenido, decisivo en móviles.

- **Limitaciones:** las sombras dependen de mapas con resolución finita que producen bordes dentados o *acné*; los reflejos en pantalla (SSR) fallan cuando lo reflejado no está visible; la iluminación indirecta suele precalcularse; la transparencia exige ordenar objetos y genera overdraw.

7.2 Ray Tracing

- **Ventajas:** reflejos, refracciones y sombras físicamente coherentes; iluminación global sin precálculo; un único modelo conceptual para todos los efectos de luz; escalabilidad en calidad añadiendo más muestras.
- **Limitaciones:** costo computacional muy superior; ruido cuando se usan pocas muestras, que obliga a usar *denoisers*; necesidad de mantener estructuras de aceleración; soporte de hardware restringido en dispositivos móviles y de gama media.

En la práctica la industria ha adoptado un **enfoque híbrido**: la imagen base se rasteriza y el trazado de rayos se reserva para efectos puntuales —reflejos, sombras o iluminación global— en las GPU que lo soportan.

8. Aplicaciones industriales y casos de uso

Tabla 2. Aplicaciones de cada paradigma.

Sector	Técnica predominante	Uso típico
Videojuegos y apps móviles	Rasterización (híbrido en PC/consola de gama alta)	Mundos interactivos a 30–120 FPS; efectos RT opcionales en PC.
Cine y animación	Path tracing fuera de línea	Fotogramas de calidad final que pueden tardar minutos u horas cada uno.
Arquitectura y diseño de producto	Path tracing y rasterización en tiempo real	Visualización fotorrealista para clientes y recorridos interactivos.
Ingeniería, óptica y acústica	Trazado de rayos físico	Simulación de lentes, iluminación de recintos, propagación de señales.
Realidad virtual y aumentada	Rasterización	Latencia mínima y dos vistas por frame a 72–120 Hz.
Visualización científica y médica	Ray marching / volumen	Tomografías, fluidos y datos volumétricos.

Cenote Vivo pertenece a la primera categoría. Su objetivo es funcionar con fluidez en teléfonos Android de gama media, por lo que se basa por completo en rasterización y simula con aproximaciones baratas los efectos que en un trazador de rayos serían naturales: la refracción del agua se obtiene desplazando la textura de la escena opaca y los reflejos provienen de un *reflection probe*.

9. Relación entre calidad visual y costo computacional

En rasterización el costo de un frame puede modelarse, de forma simplificada, como

$$T \approx T_CPU(\text{draw calls, cambios de estado}) + T_vértices(N_v \cdot c_vs) + T_fragmentos(N_px \cdot \text{overdraw} \cdot c_fs) + T_memoria(\text{ancho de banda})$$

donde N_v es el número de vértices, c_vs el costo del vertex shader, N_px el número de píxeles y c_fs el costo por fragmento. Duplicar la resolución de 1280×720 a 2560×1440 cuadruplica N_px; por eso los móviles con pantallas muy densas suelen renderizar a una resolución interna menor y escalar el resultado.

En trazado de rayos el costo crece aproximadamente como $N_px \cdot spp \cdot k \cdot \log(N_tri)$, siendo *spp* las muestras por píxel y *k* los rebotes. El error de la estimación de Monte Carlo disminuye solo con la raíz cuadrada de las muestras: para reducir el ruido a la mitad hay que **cuadruplicar** los rayos. Esta relación explica por qué el *path tracing* en tiempo real depende de los denoisers y del escalado temporal.

En el hardware, la calidad visual compite por tres recursos: **capacidad de cómputo** (ALU para shaders), **ancho de banda de memoria** (leer texturas y escribir el framebuffer) y **energía**. En un teléfono el tercero es determinante: una GPU móvil trabajando al máximo se calienta, el sistema reduce su frecuencia (*thermal throttling*) y la tasa de cuadros cae aunque la escena no haya cambiado. De ahí que las optimizaciones del proyecto —LOD, batching, texturas ASTC— busquen reducir trabajo y no solo «hacer que quepa».

10. Aliasing y técnicas de Anti-Aliasing

El **aliasing** es el error que aparece al representar una señal continua con un número finito de muestras. Una pantalla muestrea la escena en una cuadrícula regular: si la escena contiene detalles más finos que la mitad de la frecuencia de muestreo (el límite de Nyquist), esos detalles no desaparecen sino que se disfrazan de patrones falsos. Se manifiesta de tres formas:

- **Aliasing geométrico:** bordes en «escalera» o dientes de sierra en las siluetas de los objetos.

- **Aliasing de sombreado y de texturas:** brillos especulares que parpadean y patrones de *moiré* en texturas finas vistas de lejos.
- **Aliasing temporal:** objetos delgados, como las lianas del cenote, que aparecen y desaparecen al moverse la cámara (*crawling* y *shimmering*).

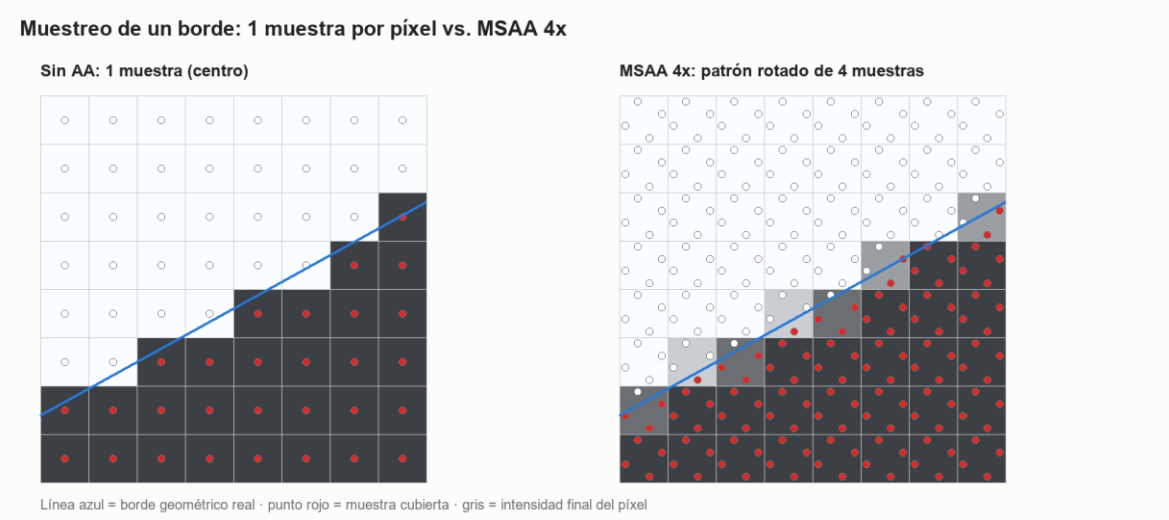


Figura 3. Un borde muestreado con una sola muestra por píxel produce una escalera; con cuatro muestras los píxeles del borde obtienen intensidades intermedias.

Las técnicas de anti-aliasing atacan el problema tomando más información por píxel o filtrando la imagen resultante:

Tabla 3. Principales técnicas de anti-aliasing.

Técnica	Principio	Corrige bien	Inconvenientes
SSAA (supersampling)	Renderiza a mayor resolución y reduce.	Todo tipo de aliasing.	Costo proporcional a las muestras (4× = 4× trabajo).
MSAA	Varias muestras de cobertura y profundidad por píxel; el shader se ejecuta una vez por píxel.	Bordes geométricos.	No corrige el aliasing de sombreado ni de transparencias recortadas; más memoria de framebuffer.
FXAA	Filtro de post-proceso que detecta contrastes de luminancia y difumina los bordes.	Bordes en general, a costo mínimo.	Suaviza también texturas y texto; no aporta información nueva.
SMAA	Detección morfológica de patrones de borde con mezcla más precisa.	Bordes, con menos desenfoque que FXAA.	Varias pasadas; algo más caro que FXAA.

Técnica	Principio	Corrige bien	Inconvenientes
TAA	Desplaza la cámara subpíxel cada frame (*jitter*) y acumula frames anteriores reproyectados con vectores de movimiento.	Bordes, sombreado y parpadeo temporal.	Estelas (*ghosting*) y desenfoque en movimiento; requiere historial y vectores de movimiento.

11. Impacto del Anti-Aliasing en el ancho de banda y los FPS

Cada técnica traslada el costo a un recurso diferente. En **MSAA** el framebuffer almacena *n* muestras de color y profundidad por píxel. A 1600×900 con color RGBA8 y profundidad de 32 bits, un framebuffer sin AA ocupa unos 11 MB; con MSAA 4× ocupa unos 44 MB antes de resolverse. En una GPU de escritorio con memoria dedicada eso implica más tráfico de lectura y escritura. En las GPU móviles basadas en teselas, en cambio, las muestras viven en la memoria del chip y se resuelven antes de escribir a la RAM, por lo que MSAA 4× suele costar poco; esa es la razón por la que fabricantes como Arm lo recomiendan como el anti-aliasing por defecto en móviles.

FXAA y **SMAA** no aumentan el tamaño del framebuffer, pero obligan a renderizar la escena en una textura intermedia y a ejecutar una o varias pasadas a pantalla completa. En móviles cada pasada completa significa leer y escribir toda la imagen en la memoria principal, justamente la operación que la arquitectura por teselas intenta evitar. **TAA** añade además un búfer de historial y uno de vectores de movimiento, por lo que es la técnica con mayor consumo de memoria y ancho de banda de las de post-proceso.

Las mediciones del prototipo en el *Entregable 2* confirman esta tendencia: en la build de prueba MSAA 4× añadió alrededor de 35 MB de memoria gráfica y 0.08 ms de GPU, mientras que activar FXAA o TAA elevó el número de render textures de 7 a más de 20 y fue la opción con mayor tiempo de GPU, porque habilita la cadena completa de post-proceso de URP.

12. Renderizado en tiempo real vs. pre-renderizado

Tabla 4. Tiempo real frente a renderizado fuera de línea.

Criterio	Tiempo real	Pre-renderizado (offline)
Presupuesto por imagen	16.6 ms (60 FPS) o 33.3 ms (30 FPS).	Segundos, minutos u horas por fotograma.

Criterio	Tiempo real	Pre-renderizado (offline)
Interactividad	El usuario controla la cámara y los eventos.	Secuencia fija decidida de antemano.
Técnica típica	Rasterización con aproximaciones e híbridos RT.	Path tracing con miles de muestras por píxel.
Hardware	Una GPU de consumo o un teléfono.	Granjas de render con cientos de nodos.
Calidad	Limitada por el presupuesto de tiempo; muchos efectos se precalculan (lightmaps).	Fotorrealista; se prioriza la exactitud física.
Ejemplos	Videojuegos, simuladores, RV, Cenote Vivo.	Cine de animación, efectos visuales, publicidad de producto.

Ambos mundos se están acercando: los motores en tiempo real incorporan trazado de rayos e iluminación global dinámica, y las producciones de cine usan motores de juego para previsualizar y para producciones virtuales. Aun así, el criterio de diseño sigue siendo el presupuesto de tiempo. Una aplicación educativa como *Cenote Vivo* debe responder al instante a cada toque, por lo que su calidad visual se construye con técnicas de tiempo real elegidas por su relación entre impacto visual y costo.

Conclusiones

La rasterización y el trazado de rayos no compiten por ser «mejores» en abstracto; resuelven el mismo problema con prioridades distintas. La rasterización domina las aplicaciones interactivas porque convierte el renderizado en un flujo masivamente paralelo y predecible, ideal para el hardware de consumo y, sobre todo, para los teléfonos. El trazado de rayos ofrece una descripción más fiel del comportamiento de la luz y se impone donde el tiempo no es una restricción o donde existe hardware dedicado.

Comprender el costo de cada etapa permite tomar decisiones informadas. En *Cenote Vivo* esas decisiones se tradujeron en un shader de agua que imita la refracción y la absorción sin trazar rayos, en mallas con niveles de detalle que reducen el trabajo de vértices y en la elección de un anti-aliasing adecuado para la arquitectura de las GPU móviles.

Referencias

Akenine-Möller, T., Haines, E., Hoffman, N., Pesce, A., Iwanicki, M. y Hillaire, S. (2018). *Real-Time Rendering* (4.ª ed.). CRC Press.

Pharr, M., Jakob, W. y Humphreys, G. (2023). *Physically Based Rendering: From Theory to Implementation* (4.ª ed.). MIT Press.

Kajiya, J. T. (1986). The rendering equation. *ACM SIGGRAPH Computer Graphics*, 20(4), 143–150.

Whitted, T. (1980). An improved illumination model for shaded display. *Communications of the ACM*, 23(6), 343–349.

Möller, T. y Trumbore, B. (1997). Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1), 21–28.

Lottes, T. (2009). *FXAA* (white paper). NVIDIA Corporation.

Karis, B. (2014). High-quality temporal supersampling. *Advances in Real-Time Rendering in Games*, SIGGRAPH 2014.

Arm Ltd. (2023). *Arm GPU Best Practices Developer Guide*. Arm Developer.

Unity Technologies. (2025). *Universal Render Pipeline: Anti-aliasing*. Documentación de Unity 6.